

RHCSA EX200

Guia Completo · Dia 07/05

Systemd Timers · Cron · at · Chrony · Tuned · Sysctl · Journald · Processos

Explicação passo a passo de TODAS as questões dos 6 Simulados

BLOCO 1 · SYSTEMD TIMERS

Um Systemd Timer funciona sempre em par: um arquivo **.service** define o *que* executar, e um arquivo **.timer** define *quando* executar. Ambos ficam em **/etc/systemd/system/**. A ordem de ativação é sempre: **daemon-reload** → **enable** → **now**.

DESTAQUE: Systemd Timers substituem o cron em ambientes modernos. No RHEL 10 / EX200, esse tema aparece em TODOS os 6 simulados. Decore o template.

Simulado 1 — Questão 7 · Timer a cada 1 minuto

Tarefa: Criar um script que acrescenta a data atual em **/var/log/system-check.log**. O timer deve disparar a cada 1 minuto.

Passo 1 — Criar o script executável

O script precisa existir em **/usr/local/bin/** e ter permissão de execução. O comando **'date'** imprime a data/hora atual; **'>>'** anexa ao arquivo de log sem apagar o conteúdo anterior.

```
#!/bin/bash
echo '#!/bin/bash' > /usr/local/bin/system-check.sh
echo 'date >> /var/log/system-check.log' >> /usr/local/bin/system-check.sh
chmod +x /usr/local/bin/system-check.sh
```

Passo 2 — Criar o arquivo **.service**

Type=oneshot indica que o serviço executa o comando e encerra (não fica em segundo plano). ExecStart aponta para o script criado.

```
cat > /etc/systemd/system/system-check.service << 'EOF'
[Unit]
Description=System Check Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/system-check.sh
EOF
```

Passo 3 — Criar o arquivo **.timer**

OnCalendar=minutely dispara no início de cada minuto (equivalente a ***-*-* *:00**). A diretiva Unit= vincula este timer ao **.service** correto.

```
cat > /etc/systemd/system/system-check.timer << 'EOF'
[Unit]
```

```
Description=Run System Check every minute
[Timer]
OnCalendar=minutely
Unit=system-check.service
[Install]
WantedBy=timers.target
EOF
```

Passo 4 — Ativar o timer (ORDEM OBRIGATÓRIA)

daemon-reload faz o systemd ler os novos arquivos. Sem ele, o enable --now não encontrará as units. list-timers confirma que o timer está ativo e mostra a próxima execução.

```
systemctl daemon-reload
systemctl enable --now system-check.timer
systemctl list-timers | grep system-check
```

■ **ATENCAO:** daemon-reload ANTES de enable --now. Sem essa ordem, o systemd não enxerga os arquivos novos.

Simulado 2 — Questão 16 · Timer diário (meia-noite)

Tarefa: Script que deleta arquivos vazios em /tmp. Timer: executa todo dia à meia-noite.

Passo 1 — Criar o script

find com -type f -empty -delete localiza arquivos regulares vazios e os remove.

```
echo '#!/bin/bash' > /usr/local/bin/clean-tmp.sh
echo 'find /tmp -type f -empty -delete' >> /usr/local/bin/clean-tmp.sh
chmod +x /usr/local/bin/clean-tmp.sh
```

Passo 2 — Criar .service e .timer

OnCalendar=daily dispara à 00:00 de cada dia. Persistent=true garante que, se o servidor estava desligado à meia-noite, a tarefa rodará imediatamente ao ligar — vantagem que o cron não tem.

```
# .service
cat > /etc/systemd/system/clean-tmp.service << 'EOF'
[Unit]
Description=Clean Empty Tmp Files
[Service]
Type=oneshot
ExecStart=/usr/local/bin/clean-tmp.sh
EOF

# .timer
cat > /etc/systemd/system/clean-tmp.timer << 'EOF'
[Unit]
Description=Run Clean Tmp Daily
[Timer]
OnCalendar=daily
Persistent=true
Unit=clean-tmp.service
```

```
[Install]
WantedBy=timers.target
EOF
```

Passo 3 — Ativar

```
systemctl daemon-reload
systemctl enable --now clean-tmp.timer
```

■ **DICA:** *Persistent=true: se o servidor estava desligado no horário agendado, o job roda imediatamente quando o servidor volta. Cron simplesmente pula a execução.*

Simulado 3 — Questão 5 · Timer a cada 2 horas

Tarefa: Script que loga 'Break Time!' em /tmp/break.log. Timer: executa a cada 2 horas.

Passo 1 — Criar o script

echo com >> anexa o texto ao arquivo de log sem sobrescrever o conteúdo existente.

```
echo '#!/bin/bash' > /usr/local/bin/break-time.sh
echo 'echo "Break Time!" >> /tmp/break.log' >> /usr/local/bin/break-time.sh
chmod +x /usr/local/bin/break-time.sh
```

Passo 2 — Criar .service e .timer com repetição periódica

Para repetição periódica usa-se a combinação OnBootSec + OnUnitActiveSec. OnBootSec=1m define que o primeiro disparo ocorre 1 minuto após o boot (inicia o ciclo). OnUnitActiveSec=2h define que, após cada execução, espera 2h para disparar novamente. Sem OnBootSec, o ciclo nunca começaria pois o serviço nunca teria rodado antes.

```
# .service
cat > /etc/systemd/system/break-time.service << 'EOF'
[Unit]
Description=Break Time Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/break-time.sh
EOF

# .timer
cat > /etc/systemd/system/break-time.timer << 'EOF'
[Unit]
Description=Run Break Time every 2 hours
[Timer]
OnBootSec=1m
OnUnitActiveSec=2h
Unit=break-time.service
[Install]
WantedBy=timers.target
EOF
```

Passo 3 — Ativar

```
systemctl daemon-reload
```

```
systemctl enable --now break-time.timer
```

■ **ATENCAO:** OnBootSec=1m é obrigatório quando se usa OnUnitActiveSec. Sem ele, o contador de 'desde a última execução' nunca começa e o timer fica inativo.

Simulado 4 — Questão 5 · Timer às 07:00 todo dia

Tarefa: Script que usa 'logger' para gravar 'Time for Break!' no syslog. Timer: todo dia às 07:00.

Passo 1 — Criar o script com logger

O comando 'logger' envia mensagens diretamente para o syslog (journald), sem precisar redirecionar para um arquivo.

```
echo '#!/bin/bash' > /usr/local/bin/notify.sh
echo 'logger "Time for Break!"' >> /usr/local/bin/notify.sh
chmod +x /usr/local/bin/notify.sh
```

Passo 2 — Criar .service e .timer com horário fixo

OnCalendar=*-*-* 07:00:00 significa: qualquer ano (*), qualquer mes (*), qualquer dia (*), às 07:00:00. Equivale ao cron: 0 7 * * *

```
# .service
cat > /etc/systemd/system/notify.service << 'EOF'
[Unit]
Description=Notify Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/notify.sh
EOF

# .timer
cat > /etc/systemd/system/notify.timer << 'EOF'
[Unit]
Description=Run Notify at 7AM
[Timer]
OnCalendar=*-*-* 07:00:00
Unit=notify.service
[Install]
WantedBy=timers.target
EOF
```

Passo 3 — Ativar e verificar

```
systemctl daemon-reload
systemctl enable --now notify.timer
systemctl list-timers | grep notify
```

■ **DICA:** OnCalendar=*-*-* 07:00:00 e OnCalendar=daily são diferentes! daily = 00:00. Para outro horário, use sempre o formato completo.

Simulado 5 — Questão 16 · Timer às 16:15 todo dia

Tarefa: Script que deleta arquivos vazios em /tmp. Timer: todo dia às 16:15.

Passo 1 — Criar script, .service e .timer

Mesmo padrão dos exercícios anteriores, apenas com horário diferente no OnCalendar.

```
echo '#!/bin/bash' > /usr/local/bin/clean_tmp.sh
echo 'find /tmp -type f -empty -delete' >> /usr/local/bin/clean_tmp.sh
chmod +x /usr/local/bin/clean_tmp.sh

# .service e .timer com OnCalendar=*-*-* 16:15:00
# (mesma estrutura dos exemplos anteriores)
```

Passo 2 — Validar a sintaxe do OnCalendar antes de criar o arquivo

O comando `systemd-analyze calendar` valida a expressão e mostra a próxima execução prevista. Use no exame para evitar erros de digitação.

```
systemd-analyze calendar "*-*-* 16:15:00"
# Saida esperada:
# Normalized form: *-*-* 16:15:00
# Next elapse: [data e hora da próxima execução]
```

```
systemctl daemon-reload
systemctl enable --now clean-tmp.timer
```

DICA DO EXAME: `systemd-analyze calendar 'expressao'` é uma ferramenta poderosa no exame: valida a sintaxe e mostra exatamente quando o timer vai disparar.

Simulado 6 — Questão 19 · Timer às 16:15 (cleanup.timer)

Tarefa: Script `/usr/local/bin/cleanup.sh` que deleta arquivos vazios em `/tmp`. Timer `cleanup.timer`: todo dia às 16:15. Mesma lógica do Sim5, nome diferente.

```
echo '#!/bin/bash' > /usr/local/bin/cleanup.sh
echo 'find /tmp -type f -empty -delete' >> /usr/local/bin/cleanup.sh
chmod +x /usr/local/bin/cleanup.sh

cat > /etc/systemd/system/cleanup.service << 'EOF'
[Unit]
Description=Cleanup Tmp Files
[Service]
Type=oneshot
ExecStart=/usr/local/bin/cleanup.sh
EOF

cat > /etc/systemd/system/cleanup.timer << 'EOF'
[Unit]
Description=Run Cleanup Daily at 16:15
[Timer]
OnCalendar=*-*-* 16:15:00
Unit=cleanup.service
[Install]
WantedBy=timers.target
EOF

systemctl daemon-reload
```

```
systemctl enable --now cleanup.timer
```

BLOCO 2 · CRON — crontab e controle de acesso

O cron é o agendador clássico do Linux. Cada linha do crontab segue o formato: **MIN HORA DIA MES DIA_SEMANA COMANDO**. O controle de acesso é feito pelos arquivos `/etc/cron.allow` e `/etc/cron.deny`.

Campo	Faixa	Exemplo
MIN	0-59	45 = minuto 45
HORA	0-23	0 = meia-noite, 7 = 07h
DIA	1-31	* = todo dia
MES	1-12	* = todo mês
DIA_SEMANA	0-7 (0=Dom)	* = qualquer

Simulado 3 — Questão 21 · Cron às 12:45 AM para root

Tarefa: Cron job para root: rodar todo dia às 12:45 AM. Comando: encontrar e deletar arquivos vazios em `/tmp`.

Passo 1 — Entender o horário

12:45 AM no formato 12h = 00:45 no formato 24h. Portanto: MIN=45, HORA=0 (meia-noite é hora 0 em formato 24h).

Passo 2 — Editar o crontab do root e adicionar a linha

`crontab -e -u root` abre o editor de crontab especificamente do usuário root. Ao salvar, o `crond` aplica as mudanças automaticamente.

```
crontab -e -u root

# Adicionar a linha abaixo e salvar:
45 0 * * * find /tmp -type f -empty -delete

# Verificar
crontab -l -u root
```

■ **DICA:** 45 0 = minuto 45, hora 0. Lembre: meia-noite é hora 0, não hora 24. 12:45 AM = 00:45 = '45 0' no crontab.

Simulado 2 — Questão 25 · Controle de acesso ao cron

Tarefa: Configurar que APENAS o usuário 'tom' pode criar cron jobs. Todos os outros devem ser negados.

Passo 1 — Entender a regra de precedência

Se `/etc/cron.allow` existe: SOMENTE os usuários listados nele podem usar cron. Todos os demais são negados automaticamente, sem precisar de `/etc/cron.deny`. Se `/etc/cron.allow` NÃO existe, verifica `/etc/cron.deny` (listados são negados, demais podem). Se nenhum existe: todos os usuários podem usar cron.

Passo 2 — Criar `/etc/cron.allow` com apenas 'tom'

O arquivo deve conter apenas o nome do usuário, um por linha.

```
# Criar o arquivo com apenas tom
echo 'tom' > /etc/cron.allow

# Verificar o conteúdo
cat /etc/cron.allow

# Saida: tom

# Testar como outro usuario (deve ser negado)
su - sam

crontab -l
```

```
# Saida: You (sam) are not allowed to use this program.
```

■ **ATENCAO:** A presença de `/etc/cron.allow` é suficiente. NAO crie `/etc/cron.deny`. Qualquer usuário fora do `cron.allow` é negado automaticamente.

BLOCO 3 · at — Tarefa única agendada

O comando **at** agenda tarefas que rodam **uma única vez** em horário específico. Diferente do cron (recorrente), at é ideal para ações pontuais: backups, manutenções, etc. O serviço **atd** precisa estar em execução.

Simulado 4 — Questão 29 · Agendar tarefa com at às 23:30

Tarefa: Usar at para agendar para 23:30 de hoje a criação de /var/tmp/late_night_check.txt. Verificar que está na fila.

Passo 1 — Garantir que o serviço atd está rodando

atd é o daemon que processa as tarefas agendadas pelo at. Se não estiver instalado, instale com dnf.

```
systemctl status atd  
  
# Se não estiver instalado:  
dnf install at -y  
systemctl enable --now atd
```

Passo 2 — Agendar a tarefa

Ao rodar 'at 23:30', o shell muda para o prompt 'at>'. Digite o comando desejado e pressione Ctrl+D para confirmar e sair.

```
at 23:30  
at> touch /var/tmp/late_night_check.txt  
at> [Ctrl+D para salvar]  
# Saída: job 1 at [data] 23:30
```

Passo 3 — Verificar a fila de tarefas

atq lista todas as tarefas pendentes. atrm N cancela a tarefa de número N.

```
atq  
  
# Saída: 1 [data] 23:30 a root
```

■ **DICA:** at = uma vez. cron = recorrente. Leia o enunciado com atenção — o exame pode pedir um ou o outro dependendo da necessidade.

BLOCO 4 · Chrony / NTP — timedatectl + chronyd

Chrony é a implementação de NTP (Network Time Protocol) usada no RHEL. O daemon **chronyd** sincroniza o relógio do sistema com servidores de tempo. **timedatectl** é a interface de linha de comando para configurar fuso horário e NTP.

Sequência padrão (igual em todos os simulados):

```
timedatectl set-timezone TIMEZONE
dnf install chrony -y
systemctl enable --now chronyd
timedatectl set-ntp true
timedatectl # verificar: NTP synchronized: yes
```

Simulado 1 — Questão 4 · Timezone: America/New_York

Tarefa: Definir timezone para America/New_York. Instalar e habilitar Chrony. Garantir sincronizacao NTP.

Passo 1 — Definir o fuso horário

timedatectl set-timezone atualiza o link simbólico /etc/localtime para o timezone correto. Para listar todos: timedatectl list-timezones | grep America

```
timedatectl set-timezone America/New_York
```

Passo 2 — Instalar e habilitar o Chrony

dnf install chrony garante que o pacote está presente. enable --now habilita no boot E inicia imediatamente.

```
dnf install chrony -y
systemctl enable --now chronyd
```

Passo 3 — Ativar sincronizacao NTP e verificar

set-ntp true instrui o timedatectl a usar o chronyd para sincronização. A saída do timedatectl deve mostrar 'NTP synchronized: yes' e o timezone correto.

```
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: America/New_York
# NTP synchronized: yes
```

Simulado 2 — Questão 4 · Timezone: Europe/London

Tarefa: Definir timezone para Europe/London. Instalar e habilitar Chrony. Garantir sincronizacao NTP.

Passo 1 — Definir o fuso horário

timedatectl set-timezone atualiza o link simbólico /etc/localtime para o timezone correto. Para listar todos: timedatectl list-timezones | grep America

```
timedatectl set-timezone Europe/London
```

Passo 2 — Instalar e habilitar o Chrony

dnf install chrony garante que o pacote está presente. enable --now habilita no boot E inicia imediatamente.

```
dnf install chrony -y
systemctl enable --now chronyd
```

Passo 3 — Ativar sincronizacao NTP e verificar

set-ntp true instrui o timedatectl a usar o chronyd para sincronização. A saída do timedatectl deve mostrar 'NTP synchronized: yes' e o timezone correto.

```
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: Europe/London
# NTP synchronized: yes
```

Simulado 4 — Questão 6 · Timezone: America/Cancun

Tarefa: Definir timezone para America/Cancun. Instalar e habilitar Chrony. Garantir sincronizacao NTP.

Passo 1 — Definir o fuso horário

timedatectl set-timezone atualiza o link simbólico /etc/localtime para o timezone correto. Para listar todos: timedatectl list-timezones | grep America

```
timedatectl set-timezone America/Cancun
```

Passo 2 — Instalar e habilitar o Chrony

dnf install chrony garante que o pacote está presente. enable --now habilita no boot E inicia imediatamente.

```
dnf install chrony -y
systemctl enable --now chronyd
```

Passo 3 — Ativar sincronizacao NTP e verificar

set-ntp true instrui o timedatectl a usar o chronyd para sincronização. A saída do timedatectl deve mostrar 'NTP synchronized: yes' e o timezone correto.

```
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: America/Cancun
# NTP synchronized: yes
```

Simulado 5 — Questão 5 · Timezone: America/Los_Angeles

Tarefa: Definir timezone para America/Los_Angeles. Instalar e habilitar Chrony. Garantir sincronizacao NTP.

Passo 1 — Definir o fuso horário

timedatectl set-timezone atualiza o link simbólico /etc/localtime para o timezone correto. Para listar todos: timedatectl list-timezones | grep America

```
timedatectl set-timezone America/Los_Angeles
```

Passo 2 — Instalar e habilitar o Chrony

dnf install chrony garante que o pacote está presente. enable --now habilita no boot E inicia imediatamente.

```
dnf install chrony -y
systemctl enable --now chronyd
```

Passo 3 — Ativar sincronizacao NTP e verificar

set-ntp true instrui o timedatectl a usar o chronyd para sincronização. A saída do timedatectl deve mostrar 'NTP synchronized: yes' e o timezone correto.

```
timedatectl set-ntp true
```

```
timedatectl
# Deve mostrar:
# Time zone: America/Los_Angeles
# NTP synchronized: yes
```

Simulado 6 — Questão 6 · Timezone: America/Chicago

Tarefa: Definir timezone para America/Chicago. Instalar e habilitar Chrony. Garantir sincronizacao NTP.

Passo 1 — Definir o fuso horário

timedatectl set-timezone atualiza o link simbólico /etc/localtime para o timezone correto. Para listar todos: timedatectl list-timezones | grep America

```
timedatectl set-timezone America/Chicago
```

Passo 2 — Instalar e habilitar o Chrony

dnf install chrony garante que o pacote está presente. enable --now habilita no boot E inicia imediatamente.

```
dnf install chrony -y
systemctl enable --now chronyd
```

Passo 3 — Ativar sincronizacao NTP e verificar

set-ntp true instrui o timedatectl a usar o chronyd para sincronização. A saída do timedatectl deve mostrar 'NTP synchronized: yes' e o timezone correto.

```
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: America/Chicago
# NTP synchronized: yes
```

DICA: *pool.ntp.org iburst no /etc/chrony.conf acelera a sincronizacao inicial. iburst envia uma rajada de 8 pacotes ao iniciar, ao inves de 1.*

BLOCO 5 · Tuned — Perfis de desempenho

O **tuned** é um daemon que aplica otimizações de kernel automaticamente baseado em perfis predefinidos. Com um único comando, você ajusta parâmetros de CPU, disco, rede e energia para o tipo de carga de trabalho.

Perfil	Quando usar
throughput-performance	Servidores de dados, banco de dados — maximizar throughput
virtual-guest	VM rodando dentro de um hypervisor
powersave	Economizar energia — laptops, VMs ociosas
balanced	Desktop/laptop geral — equilíbrio entre performance e energia
accelerator-performance	GPU, HPC, latencia ultra-baixa — desabilita power saving states

Simulado 1 — Questão 17 · Perfil: throughput-performance

Tarefa: Instalar tuned. Aplicar perfil **throughput-performance**. Maximizar throughput em servidor de dados.

Passo 1 — Instalar, habilitar e aplicar o perfil

dnf install tuned garante que o pacote esta presente. enable --now inicia o daemon. tuned-adm profile aplica o perfil imediatamente.

```
dnf install tuned -y
systemctl enable --now tuned
tuned-adm profile throughput-performance
tuned-adm active
# Saida: Current active profile: throughput-performance
```

Simulado 2 — Questão 18 · Perfil: powersave

Tarefa: Instalar tuned. Aplicar perfil **powersave**. Economizar energia.

Passo 1 — Instalar, habilitar e aplicar o perfil

dnf install tuned garante que o pacote esta presente. enable --now inicia o daemon. tuned-adm profile aplica o perfil imediatamente.

```
dnf install tuned -y
systemctl enable --now tuned
tuned-adm profile powersave
tuned-adm active
# Saida: Current active profile: powersave
```

Simulado 5 — Questão 9 · Perfil: virtual-guest

Tarefa: Instalar tuned. Aplicar perfil **virtual-guest**. Otimizar para VM rodando em hypervisor.

Passo 1 — Instalar, habilitar e aplicar o perfil

dnf install tuned garante que o pacote esta presente. enable --now inicia o daemon. tuned-adm profile aplica o perfil imediatamente.

```
dnf install tuned -y
systemctl enable --now tuned
tuned-adm profile virtual-guest
```

```
tuned-adm active  
# Saida: Current active profile: virtual-guest
```

Simulados 3/4/6 · Combinando dois perfis (stacking)

O tuned suporta combinar (stackar) múltiplos perfis. O segundo perfil tem prioridade em configurações conflitantes.

Simulado 3 — Q11 — VM + economia de energia:

```
dnf install tuned -y && systemctl enable --now tuned  
tuned-adm profile virtual-guest powersave  
tuned-adm active  
# Saida: Current active profile: virtual-guest powersave
```

Simulado 4 — Q12 — VM + latencia ultra-baixa:

```
dnf install tuned -y && systemctl enable --now tuned  
tuned-adm profile virtual-guest accelerator-performance  
tuned-adm active  
# Saida: Current active profile: virtual-guest accelerator-performance
```

Simulado 6 — Q12 — VM + economia de energia:

```
dnf install tuned -y && systemctl enable --now tuned  
tuned-adm profile virtual-guest powersave  
tuned-adm active  
# Saida: Current active profile: virtual-guest powersave
```

■ **DICA:** tuned-adm profile P1 P2: espaço entre os perfis, não vírgula. O segundo tem prioridade em conflitos.

BLOCO 6 · Sysctl — Parâmetros do kernel

O **sysctl** permite ler e modificar parâmetros do kernel em tempo de execução. Alterações em `/proc/sys/` são temporárias (perdidas no reboot). Para persistência, use arquivos em `/etc/sysctl.d/`.

Simulados 2/3/4/5/6 · Habilitar packet forwarding IPv4 e IPv6

Tarefa: Habilitar encaminhamento de pacotes IPv4 e IPv6 de forma persistente após reboot.

Passo 1 — Entender o que é packet forwarding

Packet forwarding (IP forwarding) permite que o Linux atue como roteador/gateway, encaminhando pacotes recebidos em uma interface de rede para outra. Necessário para: VPNs, NAT, containers com rede bridge, etc.

Passo 2 — Criar arquivo dedicado em `/etc/sysctl.d/`

O prefixo numérico (99-) define a ordem de leitura. Arquivos com número maior são lidos por último e sobrescrevem configurações anteriores. Use um número alto (como 99) para garantir que sua configuração tenha prioridade.

```
vim /etc/sysctl.d/99-forwarding.conf

# Conteudo do arquivo:

net.ipv4.ip_forward = 1
net.ipv6.conf.all.forwarding = 1
```

Passo 3 — Aplicar AGORA sem reboot

`sysctl -p` aplica as configurações do arquivo especificado imediatamente. `sysctl --system` aplica todos os arquivos em `/etc/sysctl.d/` e `/etc/sysctl.conf`.

```
# Aplicar apenas o arquivo criado:
sysctl -p /etc/sysctl.d/99-forwarding.conf

# OU aplicar todos os arquivos de uma vez:
sysctl --system

# Verificar:
sysctl net.ipv4.ip_forward

# Saida: net.ipv4.ip_forward = 1
sysctl net.ipv6.conf.all.forwarding

# Saida: net.ipv6.conf.all.forwarding = 1
```

■ **ATENCAO:** Editar `/proc/sys/net/ipv4/ip_forward` diretamente funciona mas NAO persiste apos reboot. SEMPRE use `/etc/sysctl.d/` para persistencia.

BLOCO 7 · Journald — Logs persistentes

O **journald** (systemd-journald) coleta logs de todos os serviços gerenciados pelo systemd. Por padrão (Storage=auto), os logs ficam em RAM (/run/log/journal) e são perdidos no reboot. Com **Storage=persistent**, os logs são salvos em disco (/var/log/journal).

Simulado 1 — Questão 21 · Journald persistente + filtro sshd

Tarefa: Tornar journald persistente. Filtrar logs do sshd do boot atual e salvar em arquivo.

Passo 1 — Configurar Storage=persistent no journald.conf

Abra /etc/systemd/journald.conf, localize a linha #Storage=auto e altere para Storage=persistent. A linha não deve ter # (comentário) na frente.

```
vim /etc/systemd/journald.conf
# Em [Journal], alterar/adicionar:
Storage=persistent
```

Passo 2 — Reiniciar o journald e fazer flush

O restart aplica a nova configuração. --flush força a gravação imediata de logs da RAM para o disco.

```
systemctl restart systemd-journald
journalctl --flush

# Verificar que o diretorio foi criado:
ls -ld /var/log/journal
```

Passo 3 — Exportar logs do sshd do boot atual

-u sshd filtra por unidade (unit). -b filtra apenas o boot atual. -b -1 seria o boot anterior. O operador > redireciona para o arquivo.

```
journalctl -u sshd -b > /var/log/ssh_boot.log

# Verificar o resultado:
head /var/log/ssh_boot.log
```

Filtros journalctl mais usados no exame

journalctl -u sshd	Logs apenas do sshd
journalctl -b	Logs do boot atual
journalctl -b -1	Logs do boot anterior
journalctl --since '1 hour ago'	Logs da ultima hora
journalctl -u sshd -b	sshd + boot atual (combinar filtros)
journalctl --disk-usage	Espaco usado pelos logs

Simulado 5 — Questão 25 · Journald persistente com limite de 100MiB

Tarefa: Configurar journald persistente e limitar o tamanho máximo dos logs a 100MiB.

Passo 1 — Configurar persistence e limite em journald.conf

SystemMaxUse define o espaço máximo total dos logs no disco. Quando o limite é atingido, logs antigos são deletados automaticamente (rotação).

```
vim /etc/systemd/journald.conf
```

```
# Em [Journal], adicionar/descomentar:  
Storage=persistent  
SystemMaxUse=100M
```

Passo 2 — Reiniciar e verificar

```
systemctl restart systemd-journald  
  
# Verificar diretorio e uso de disco:  
ls -ld /var/log/journal  
journalctl --disk-usage
```

BLOCO 8 · Processos — nice, renice, kill, signals

O **nice** e o **renice** controlam a prioridade de CPU de processos. O valor de niceness vai de **-20** (maior prioridade) a **+19** (menor prioridade). O padrão é 0. Apenas root pode usar valores negativos.

Sinal	Número	Acao
SIGTERM	15	Encerramento gracioso — processo pode limpar recursos
SIGKILL	9	Encerramento forçado — nao pode ser ignorado
SIGINT	2	Interrupcao (equivale ao Ctrl+C)

Simulado 1 — Questão 22 · nice=5, SIGTERM

Tarefa: Iniciar sleep 1000 em background. Encontrar PID. Mudar niceness para 5. Enviar SIGTERM.

Passo 1 — Iniciar sleep 1000 em background

O & ao final envia o processo para background. O shell imprime o PID na saída. Guarde esse numero.

```
sleep 1000 &  
# Shell imprime ex: [1] 12345 (numero do job e PID)
```

Passo 2 — Encontrar o PID (caso nao tenha anotado)

pgrep busca por nome de processo. jobs -l lista jobs com PID.

```
pgrep -a sleep  
# OU:  
jobs -l
```

Passo 3 — Mudar niceness para 5

renice MUDA a niceness de um processo ja em execucao. nice iniciaria um NOVO processo com essa niceness. NAO confundir.

```
renice -n 5 -p PID  
ps -o pid,ni,comm -p PID # verificar NI={niceval}
```

Passo 4 — Enviar SIGTERM e confirmar encerramento

```
kill PID  
jobs # deve mostrar: [1]+ Terminated
```

Simulado 2 — Questão 30 · nice=15, SIGTERM

Tarefa: Iniciar sleep 5000 em background. Encontrar PID. Mudar niceness para 15. Enviar SIGTERM.

Passo 1 — Iniciar sleep 5000 em background

O & ao final envia o processo para background. O shell imprime o PID na saída. Guarde esse numero.

```
sleep 5000 &  
# Shell imprime ex: [1] 12345 (numero do job e PID)
```

Passo 2 — Encontrar o PID (caso nao tenha anotado)

pgrep busca por nome de processo. jobs -l lista jobs com PID.

```
pgrep -a sleep  
# OU:  
jobs -l
```

Passo 3 — Mudar niceness para 15

renice MUDA a niceness de um processo ja em execucao. nice iniciaria um NOVO processo com essa niceness. NAO confundir.

```
renice -n 15 -p PID
ps -o pid,ni,comm -p PID # verificar NI={niceval}
```

Passo 4 — Enviar SIGTERM e confirmar encerramento

```
kill PID
jobs # deve mostrar: [1]+ Terminated
```

Simulado 3 — Questão 25 · nice=10, SIGTERM

Tarefa: Iniciar sleep 1000 em background. Encontrar PID. Mudar niceness para 10. Enviar SIGTERM.

Passo 1 — Iniciar sleep 1000 em background

O & ao final envia o processo para background. O shell imprime o PID na saida. Guarde esse numero.

```
sleep 1000 &
# Shell imprime ex: [1] 12345 (numero do job e PID)
```

Passo 2 — Encontrar o PID (caso nao tenha anotado)

pgrep busca por nome de processo. jobs -l lista jobs com PID.

```
pgrep -a sleep
# OU:
jobs -l
```

Passo 3 — Mudar niceness para 10

renice MUDA a niceness de um processo ja em execucao. nice iniciaria um NOVO processo com essa niceness. NAO confundir.

```
renice -n 10 -p PID
ps -o pid,ni,comm -p PID # verificar NI={niceval}
```

Passo 4 — Enviar SIGTERM e confirmar encerramento

```
kill PID
jobs # deve mostrar: [1]+ Terminated
```

Simulado 4 — Questão 23 · Enviar SIGINT (sem renice)

Tarefa: Iniciar sleep 10000 em background. Encontrar PID. Enviar SIGINT.

Passo 1 — Iniciar e localizar

```
sleep 10000 &
# Shell imprime: [1] 12345
pgrep -a sleep # alternativa para encontrar PID
```

Passo 2 — Enviar SIGINT (sinal 2)

SIGINT equivale ao Ctrl+C. Pode ser enviado como kill -2, kill -SIGINT ou kill -INT.

```
kill -2 PID
# OU equivalente:
kill -SIGINT PID
```

```
# Confirmar:
```

```
jobs # Terminated
```

■ **ATENCAO:** nice INICIA um processo com niceness. renice MUDA niceness de processo ja rodando.
Confundir os dois e erro classico no exame.

CHECKLIST FINAL — O que dominar ao final do Dia 07/05

1	Par .service (Type=oneshot + ExecStart) e .timer (OnCalendar + Unit=)	[]
2	daemon-reload ANTES de enable --now — ordem obrigatoria	[]
3	systemctl list-timers para ver proxima execucao	[]
4	OnCalendar: minutely, daily, *-*- 07:00:00, *-*- 16:15:00	[]
5	OnBootSec=1m + OnUnitActiveSec=2h para repeticao a cada 2h	[]
6	Persistent=true para rodar job perdido ao religar o servidor	[]
7	crontab formato: MIN HORA DIA MES DIASEM COMANDO	[]
8	45 0 * * * = 12:45 AM. 0 7 * * * = 07:00	[]
9	/etc/cron.allow: so os listados podem usar cron	[]
10	at 23:30 → prompt at> → Ctrl+D para salvar. atq para verificar.	[]
11	timedatectl set-timezone + dnf install chrony + enable --now chronyd	[]
12	timedatectl set-ntp true + verificar NTP synchronized: yes	[]
13	tuned-adm profile NOME. Combinar: tuned-adm profile P1 P2	[]
14	/etc/sysctl.d/99-forwarding.conf + sysctl -p arquivo	[]
15	journald: Storage=persistent + SystemMaxUse=100M em journald.conf	[]
16	journalctl -u sshd -b > /var/log/ssh_boot.log	[]
17	nice -n N cmd & (iniciar). renice -n N -p PID (modificar)	[]
18	kill = SIGTERM(15). kill -9 = SIGKILL. kill -2 = SIGINT	[]

COMPARATIVO — Systemd Timer vs Cron vs at

Característica	Systemd Timer	Cron	at
Tipo de tarefa	Recorrente	Recorrente	Uma unica vez
Job perdido (server desligado)	Sim (Persistent=true)	Nao	Sim (roda ao ligar)
Logs	journalctl	/var/log/cron	email/syslog
Arquivo de config	/etc/systemd/system/	crontab -e	at comando
Ativar	systemctl enable --now	automatico	atd service
Cobrado no EX200	SIM — todos simulados	SIM	SIM (1 simulado)

Prova RHCSA EX200 — Dia 14/05

Bons estudos! Pratique cada comando no terminal ate sair de memoria.